# ARK Selection Task Perception Team: Medial Axis Detection of Moving Objects

Aashi Asmita

***Abstract*—**
**This project implements a computer vision method to detect the medial axis of moving objects in a video using Python and OpenCV. The input video is processed frame by frame, where background subtraction using the Mixture of Gaussian's method is applied to isolate foreground objects. Morphological operations are used to reduce noise, followed by Canny edge detection to extract object boundaries. A manual implementation of the Hough Transform is then used to detect the two most dominant lines in the edge map. The medial axis is computed as the average of these detected lines and drawn on the frame. To improve stability, the first 30 frames are excluded from the medial axis calculation to allow the background model to stabilize. The processed frames are resized and written to an output video file showing the detected medial axis throughout the sequence.**

## I. INTRODUCTION

The goal of this project is to detect the medial axis (center line) of a moving object in a video. Extracting the medial axis helps in understanding the orientation and structure of the object,

The video is processed frame by frame using Python and OpenCV. First, background subtraction (MOG2) is used to separate the moving object from the background. Morphological operations are applied to remove noise, followed by Canny edge detection to extract object edges. A manual Hough Transform is then used to detect the two dominant boundary lines. The medial axis is computed by averaging these lines and drawn on the frame. The first 30 frames are ignored to allow the background model to stabilize, and the processed frames are saved as an output video.

## II. PROBLEM STATEMENT

The objective of this task is to detect and highlight the medial axis (central skeletal line) of a moving object in a video sequence. The input consists of video clips showing a moving surgical tool, and the goal is to process these frames in order to determine the center line of the tool and overlay it on the original video frames.

The challenge lies in accurately extracting the structure of the moving object from the video despite noise, background variations, and possible motion artifacts. Since the object is moving within a scene that contains a static background, the first step is to isolate the foreground object from the background. Once the foreground is extracted, the resulting image may still contain noise and irregularities that must be cleaned using appropriate image processing techniques.

After obtaining a clean silhouette of the moving tool, the next step is to detect the edges and straight boundaries of the object. Edge detection techniques based on image derivatives are used to identify significant intensity changes that correspond to object boundaries. From these edges, the dominant straight lines representing the sides of the tool are identified using the Hough Line Transform.

A key constraint of this task is that the Hough Transform must be implemented manually, without using OpenCV's built-in function. This requirement ensures a deeper understanding of how line detection works through the voting process in the parameter space defined by distance and angle.

Once the main boundary lines of the tool are detected, the medial axis is computed by estimating the center line between these detected edges. This axis represents the geometric center of the object along its length. Finally, the computed medial axis is drawn and overlaid onto the original video frames to visually highlight the central skeletal structure of the moving tool.

The final output of the system is a processed video in which the detected medial axis is continuously displayed on the original frames, demonstrating the successful application of background subtraction, image cleaning, edge detection, manual Hough line detection, and geometric analysis to identify the central axis of a moving object.

## III. RELATED WORK

Besides the implemented approach, several other methods could be used to detect the medial axis of a moving object. One possible method is skeletonization, where the binary silhouette of the object is thinned iteratively until a one-pixel-wide centerline representing the medial axis is obtained. Another approach is the distance transform method, which computes the distance of each pixel from the nearest boundary and extracts the ridge points to form the medial axis.

Additionally, probabilistic Hough Transform or RANSAC-based line fitting could be used to detect dominant lines more robustly in noisy edge maps. Machine learning or deep learning based segmentation methods could also be applied to accurately extract the object before computing its centerline. These methods may provide improved robustness but often require higher computational cost or additional training data.

## IV. INITIAL ATTEMPTS

Faster Manual Hough improves the initial Manual Hough Transform mainly in terms of speed and efficiency.

Key differences:

Reduced Computation

Initial manual Hough checks all possible angles () for every edge pixel, which is slow.

Faster version limits the angle range or step size, reducing calculations.

Edge Filtering

Faster method may use fewer but stronger edge points.

This decreases the number of votes in the accumulator.

Optimized Voting

Instead of brute-force voting for many values, it may use vectorized operations or precomputed values.

Less Memory Usage

Smaller accumulator space or fewer parameter combinations.

Result:

Same or similar line detection accuracy

Much faster execution time compared to the initial manual implementation.

## V. Final Approach

This method processes a video to detect and draw a prominent line using a faster manual implementation of the Hough Transform. It begins by importing the OpenCV and NumPy libraries, which are used for video processing and numerical operations respectively. The program opens a video file named "1.mp4" and prepares to process it frame by frame. At the same time, a video writer object is created to save the processed frames into a new output file called "output.mp4". The output video is set to run at 30 frames per second with a resolution of 700 × 400 pixels, and it uses the mp4v codec to encode the video.

To focus on relevant motion in the scene, the program applies a background subtraction technique using the MOG2 algorithm. This method separates moving objects from the static background by creating a foreground mask. Because raw foreground masks often contain noise and small unwanted regions, the program applies morphological operations to clean the mask. Specifically, it performs an opening operation to remove small noise particles and a closing operation to fill small holes inside detected objects. After cleaning the mask, the program performs Canny edge detection, which extracts the edges from the foreground regions. These edges represent the boundaries of moving structures and will later be used for line detection.

A frame counter is used to keep track of how many frames have been processed. The program waits until the first 30 frames have passed before performing line detection because the background subtractor requires a few frames to learn the background model properly. Before the main loop starts, the program also precomputes sine and cosine values for angles from 0 to 179 degrees. This optimization speeds up the Hough Transform calculations because the trigonometric values do not need to be recalculated for every pixel during each frame.

Once the frame count exceeds 30, the program performs a faster manual Hough Transform on the detected edge pixels. First, it determines the dimensions of the edge image and calculates the maximum possible rho value based on the diagonal length of the frame. An accumulator matrix is then created to store votes for different combinations of rho and theta values, where each cell represents a possible line in the image. The program extracts the coordinates of all edge pixels and processes them one by one. For each edge pixel, it computes the rho values for all possible theta angles using the precomputed sine and cosine arrays. These rho values are rounded, shifted to fit the accumulator index range, and then used to add votes to the accumulator. This approach uses NumPy vector operations instead of nested loops, which makes the manual Hough Transform significantly faster.

After all edge pixels have contributed their votes, the accumulator matrix contains peaks that correspond to strong lines in the image. The program flattens the accumulator and identifies the two highest peaks, which represent the two most dominant lines detected in the frame. These peak indices are converted back into rho and theta values. If two lines are found, the program calculates the midpoint between them by averaging their rho and theta values. This produces a single representative line located between the two detected lines.

Before drawing the line, the program checks whether the two detected lines are nearly parallel by comparing their angle difference. If the difference is less than five degrees, the lines are considered parallel and likely represent the two sides of the same structure. In this case, the midpoint line parameters are stored and a red line is drawn on the frame using the computed rho and theta values. Finally, the frame is resized to match the output resolution and written to the output video file.

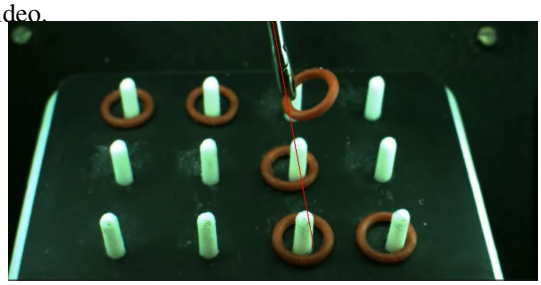
This process continues for every frame until the video ends. After the loop finishes, the program releases the video capture and video writer objects and closes any OpenCV resources. Overall, the program performs motion-based edge extraction followed by a faster manual Hough Transform to detect prominent lines in moving regions of a video and visualize them in the output video.

## VI. Results and Observation

The program successfully processes the input video and detects dominant lines in moving regions using a faster manual implementation of the Hough Transform. Background subtraction isolates moving objects, edge detection extracts their boundaries, and the Hough voting process identifies the strongest lines. The midpoint of the two most prominent and nearly parallel lines is computed and drawn on each frame.

The processed frames are saved into an output video with the detected line highlighted.

The use of background subtraction helps reduce unnecessary computations by focusing only on moving areas. Morphological operations effectively remove noise from the foreground mask, improving edge detection accuracy. Precomputing sine and cosine values and using vectorized NumPy operations significantly speeds up the manual Hough Transform compared to a basic implementation. Additionally, checking for parallel lines and drawing their midpoint helps produce a stable and meaningful line representation in the video.



## CONCLUSION

This task demonstrates how classical computer vision techniques can be combined to detect structural features in a video. By applying background subtraction, morphological filtering, edge detection, and a faster manual implementation of the Hough Transform, dominant lines in moving objects can be effectively identified. The midpoint of the detected parallel lines provides a reliable representation of the object's central axis. The approach improves computational efficiency through precomputed trigonometric values and vectorized operations, making the manual Hough Transform significantly faster while maintaining accurate line detection.

This method finds application in motion analysis and autonomous navigation, which plays a key role in Aerial Robotics as well.

## REFERENCES

[1] OpenCV. (2024). OpenCV-Python Tutorials. Available at: https://docs.opencv.org/